Local RAG with n8n — The Complete Guide

Build a **fully local, self-hosted Retrieval-Augmented Generation (RAG)** application — one that reads your own documents and answers questions about them — using **n8n**, **PostgreSQL**, **Qdrant**, and **Ollama**, all in Docker on your own machine. No cloud, no API keys, nothing leaves your laptop.

By the end you'll have set up the stack, ingested a real document (we use Andrew Ng's book *Machine Learning Yearning*), chatted with it, logged every question and answer to a database, and wrapped the whole thing in your own web page.

Work through it top to bottom. Each step is a short description followed by the exact command or click. Only move on when a step finishes successfully.

> 📋 ***How to read this:*** *any command in a grey box is something you* ***type into your terminal and press Enter****. Boxes marked do this in n8n are clicks in your browser, not the terminal.*

> 💡 ***What is RAG? (optional — skip if you just want to build)*** *A language model like* `llama3.2` *only knows what it was trained on; it can't see your documents. RAG fixes that in two phases.* ***Ingestion:*** *split a document into chunks and use an embedding model to turn each chunk into a list of 768 numbers (a vector) that captures its meaning, then store those vectors in a vector database.* ***Retrieval + generation:*** *embed your question the same way, ask the database for the chunks closest to it, and hand those chunks to the model as context. The model then answers grounded in your document instead of guessing. Search by meaning, then let the model write the answer from what it found.*

Before you start — two rules and two folders

You'll type addresses in two places, and the rule differs:

- **In your web browser** → use **`localhost`** (e.g. n8n at `http://localhost:5678`). Your browser runs on your laptop.
- **Inside an n8n field** (credentials and nodes) → use the service's **container name**, never `localhost` (e.g. `http://ollama:11434`). n8n runs *inside* Docker, where `localhost` would mean n8n itself. The exact address is given at each step.

Many n8n fields have a **`Fixed` / `Expression`** toggle. **Fixed** holds literal text; **Expression** evaluates `{{ ... }}` as code. Rule of thumb: **if what you type contains `{{ }}`, set the field to Expression** — otherwise n8n stores it as plain characters and your data never flows. A correct expression shows a grey preview of its value below the field.

You will move between just two folders. Each step tells you which one to be in:

- **`agentic-rag-n8n-cohort`** — the main folder (the helper scripts live here).
- **`agentic-rag-n8n-cohort/self-hosted-ai-starter-kit`** — the stack folder (Docker runs from here).

> *Lost track of where you are? Type* `pwd` *and press Enter — it prints your current folder.*

> 🆘 ***Stuck at any point?*** *See the companion* ***Troubleshooting & Tuning*** *doc — it lists the problems people hit most often, each with the exact fix.*

Part A — Set up the stack

The only thing you install by hand is **Docker Desktop**. Everything else — the pinned n8n starter kit, the helper scripts, the sample document, the SQL, and the web frontend — comes in the course repo you clone. You do **not** clone the starter kit separately, and you do **not** build any Docker images.

A1. Open the right terminal

On **Windows** you must use the **WSL2 Linux terminal** — the **Ubuntu app**, **NOT** PowerShell and **NOT** Command Prompt. Most failed setups happen because commands were typed into PowerShell. On **Mac**, the built-in **Terminal** is correct. From here on, every command is identical on both.

- **Windows:** click **Start**, type **Ubuntu**, open the **Ubuntu** app *(it may show as "Ubuntu" or "Ubuntu 24.04 LTS" — either is correct)*.
- **Mac:** press **Cmd + Space**, type **Terminal**, open it.

> *Check you're in the right place: your prompt should look like* `you@PC:~$` *(Linux),* ***not*** `PS C:\Users\you>` *(PowerShell).*

A2. Install Docker Desktop and start it

Docker Desktop runs the entire stack for you. Install it, start it, and wait until the whale icon stops animating. On **Windows** you must also turn on WSL Integration so Docker works from Ubuntu.

1. Download and install: https://www.docker.com/products/docker-desktop/
2. Start Docker Desktop; wait until the whale icon stops animating.
3. **Windows only:** Docker Desktop → **Settings → Resources → WSL Integration** → toggle **ON** your Ubuntu distro → **Apply & Restart**.

A3. Clone the course repo

This downloads everything you need into a new folder. The second line moves you inside it.

```
git clone https://github.com/SwarupSG/agentic-rag-n8n-cohort.git
cd agentic-rag-n8n-cohort
```

> *You are now inside* ***`agentic-rag-n8n-cohort`*** *. Stay here for A4.*

> 💡 ***Recommended — work inside your IDE.*** *Open the cloned folder in* ***VS Code or Antigravity (connected to WSL)*** *and use its built-in* ***Terminal*** *panel — you'll see your files on the left, and that terminal is already the WSL shell (not PowerShell). Run* `code .` *(VS Code) or open the folder in Antigravity. Prefer a plain terminal? That's fine — just keep using the WSL Ubuntu terminal (Mac: Terminal).*

> 🛑 ***Windows/WSL2 performance:*** *clone into your* ***WSL2 home*** *(e.g.* `~/agentic-rag-n8n-cohort` *),* ***not*** *under* `/mnt/c/...` *— Docker volumes on the Windows-mounted drive are dramatically slower.*

A4. Run the preflight check

This confirms your machine is ready — Docker reachable, enough disk space, correct terminal — before you start anything.

Folder: **`agentic-rag-n8n-cohort`** **.** Try this first:

```
./scripts/preflight.sh
```

Only if that says `Permission denied`, use this instead:

```
bash scripts/preflight.sh
```

Continue only when it ends with **Preflight passed**.

A5. Create the .env file

The stack needs a small settings file named `.env`. Only a template exists so far. The first line moves you into the stack folder; the second copies the template into the real file.

Folder: `agentic-rag-n8n-cohort`.

```
cd self-hosted-ai-starter-kit
cp .env.example .env
```

> *You are now inside* ***`self-hosted-ai-starter-kit`****. Stay here for A6–A8.*

> 📝 *The* `.env` *you just created already includes the file-access settings this course needs (*`N8N_RESTRICT_FILE_ACCESS_TO=/data/shared`*), so ingestion can read your document later.*

A6. Start the stack

This downloads and starts all four containers (n8n, Ollama, Qdrant, Postgres). The **first run pulls a few GB**, so give it a few minutes. Every line of output should end with `Started`.

Folder: `self-hosted-ai-starter-kit`. Run the CPU line — it works on any machine:

```
docker compose --profile cpu up -d
```

> ***Have an NVIDIA GPU?*** *Run this* ***instead*** *— do not run both:*
>
> ```
> docker compose --profile gpu-nvidia up -d
> ```

If `up` stops partway with an error (the stack is half-started), reset and try once more:

```
docker compose --profile cpu down
docker compose --profile cpu up -d
```

A7. Confirm Ollama is running

The next step reaches *into* the Ollama container, so it must be up. This lists running containers — look for `ollama`.

Folder: any (this works anywhere).

```
docker ps
```

If you don't see `ollama`, repeat A6 first — otherwise the model pulls fail with *"container … is not running."*

A8. Pull the two models

These download the AI models into the Ollama container — one for chat, one for turning text into vectors. They run *inside* the container, which is why Ollama had to be up first.

Folder: any. Run each and wait for **success**:

```
docker exec ollama ollama pull llama3.2
```

```
docker exec ollama ollama pull nomic-embed-text
```

> 💡 ***Why two models?*** `llama3.2` *writes answers;* `nomic-embed-text` *turns text into the 768-number vectors Qdrant searches. Different jobs, different models.*

A9. Verify everything

This checks the whole setup end to end — all containers up, both models present, n8n able to reach the other services. The script lives one level up, so first go back to the main folder.

Folder: `self-hosted-ai-starter-kit` → move back:

```
cd ..
```

> *You are now back in* `agentic-rag-n8n-cohort`. *Try this first:*

```
./scripts/verify_setup.sh
```

Only if that says `Permission denied`, use this instead:

```
bash scripts/verify_setup.sh
```

You're ready when it prints **All checks passed**.

A10. Open the apps

Open these in your normal web browser (these `localhost` links work from your laptop). **n8n** is where you build everything; the **Qdrant** dashboard shows your vector database.

- n8n → http://localhost:5678
- Qdrant → http://localhost:6333/dashboard

✅ **Part A done** when `verify_setup.sh` passes and the n8n page loads.

Part B — Wire up n8n (credentials + Qdrant collection)

n8n needs to know how to reach Ollama and Qdrant, and Qdrant needs a collection to store vectors in. About 10 minutes, mostly clicking.

> ***Everything here happens inside n8n.*** *There's no "Ollama" or "Qdrant" app to open — they're just credential types you pick from a list. Ollama has no screen at all; it's an API n8n talks to.*

B1. Create your n8n owner account

Open http://localhost:5678. On first launch, n8n asks you to set up an owner — enter any email + password (it's local, stored only in your Postgres). You land on the workflow canvas.

B2. Add the Ollama credential

1. Go to http://localhost:5678/home/credentials.
2. Click **Create credential** → search **Ollama** → select **Ollama account**.
3. **Base URL:** `http://ollama:11434` ← container name, *not* the pre-filled `localhost:11434`.
4. **API Key:** leave **blank**.
5. **Save.** A green "Connection tested successfully" confirms n8n can reach Ollama.

B3. Add the Qdrant credential

1. Back on the Credentials page, click **Create credential** → search **Qdrant** → select it.
2. **Qdrant URL:** `http://qdrant:6333`
3. **API Key:** the field is marked required, but local Qdrant has no auth — type any placeholder, e.g. `local`. Qdrant ignores it.
4. **Save.**

B4. Create the Qdrant collection

A collection is like a table for vectors. It must match the embedding model's size (**768** for `nomic-embed-text`) and use **Cosine** distance. Create it once from your terminal:

```
curl -X PUT http://localhost:6333/collections/documents -H 'Content-Type:
application/json' -d '{"vectors":{"size":768,"distance":"Cosine"}}'
```

Verify it exists (status `green`, points `0`):

```
curl -s http://localhost:6333/collections/documents
```

> 💡 ***Why 768?*** *`nomic-embed-text` turns each chunk into a list of 768 numbers. The collection's `size` must equal that, or inserts fail.*

✅ **Part B done** when both credentials save green and the `documents` collection shows `green`.

Part C — Ingest your document

This workflow reads a document, splits it into chunks, turns each chunk into a 768-number vector, and stores those vectors in the `documents` collection. Once stored, Part D can search them.

Node chain:

```
Manual Trigger -> Read Files -> Extract from File -> Qdrant Vector Store (Insert)
                                                      |-- Embeddings Ollama   (sub-
node)
                                                      |-- Default Data Loader (sub-
node)
```

C1. Stage your document

n8n can only read files inside its `shared/` folder (mounted into the container at `/data/shared`). The sample PDF — *Machine Learning Yearning* — already came with your clone at `sample-docs/machine-learning-yearning.pdf`. Copy it into the drop-zone:

Folder: `agentic-rag-n8n-cohort`.

```
cp sample-docs/machine-learning-yearning.pdf self-hosted-ai-starter-
kit/shared/corpus/
```

> 💡 ***Use your own document instead?*** *Drop any `.pdf`, `.txt`, `.md`, `.csv`, `.xlsx`, or `.html` into `self-hosted-ai-starter-kit/shared/corpus/` and point the Read node (C3) at it. Word `.docx` isn't supported — save as PDF first.*

C2. Create the workflow and trigger — *do this in n8n*

1. **Create** (top-left **+**) → **Workflow**. Name it `Ingest documents`.
2. Click **+** on the canvas → search **Manual Trigger** → add **When clicking 'Test workflow'**.

C3. Read the file from disk — *do this in n8n*

1. Click **+** after the trigger → search **Read/Write Files from Disk** → add it.
2. **Operation:** `Read File(s) From Disk`.
3. **File(s) Selector:** `/data/shared/corpus/machine-learning-yearning.pdf`

> ⚠️ ***Use the container path, NOT your computer's path.*** *n8n runs inside Docker, so it only sees `/data/shared/...`. A real disk path (`/Users/...` or `C:\Users\...`) gives **"No file(s) found"**. Always `/data/shared/corpus/machine-learning-yearning.pdf`.*

4. Click **Execute step** to test just this node — you should see the file's binary in OUTPUT under a `data` field. *(Don't run the whole workflow yet — the later nodes don't exist.)*

C4. Add the Extract from File node — *do this in n8n*

A PDF is binary; n8n must pull the text out before chunking.

1. Click **+** after the Read node → search **Extract from File** → add it.
2. **Operation:** `Extract From PDF`.
3. **Input Binary Field:** `data` (the default — the field the Read node outputs).

> 💡 *Plain `.txt` / `.md` skip this node — connect Read straight to Qdrant and set the Data Loader's **Type of Data** to `Binary` (see C7).*

C5. Add the Qdrant Vector Store node (insert) — *do this in n8n*

1. Click **+** after Extract → search **Qdrant Vector Store** → add it.
2. In the action picker, choose **Add documents to vector store** (shown afterward as **Insert Documents**).
3. **Credential:** your **Qdrant account**.
4. **Qdrant Collection:** `From list` → `documents`.
5. **Embedding Batch Size:** leave the default (`200`).

The node shows two required connectors — **Embedding *** and **Document *** (the `*` means required). Fill both next. *(Don't run it until both are connected.)*

C6. Attach the Embeddings sub-node — *do this in n8n*

1. On the Qdrant node, click the **Embeddings** connector → search **Embeddings Ollama** → add it.
2. **Credential:** your **Ollama account**.
3. **Model:** `nomic-embed-text:latest` ← must match the collection's 768 size.

C7. Attach the Default Data Loader sub-node — *do this in n8n*

1. On the Qdrant node, click the **Document** connector → search **Default Data Loader** → add it.
2. **Type of Data:** `JSON` ← because Extract already turned the PDF into a text field.
3. **Mode:** `Load All Input Data`.
4. **Text Splitting:** `Simple` — auto-applies chunk size 1000, overlap 200.

C8. Run it — *do this in n8n*

1. Click **Test workflow** (the trigger's play button). Each node should go green; the Qdrant node reports how many vectors it inserted.

> ⏱️ ***Expect about a minute** for this book (~35 pages of text): it becomes a couple of hundred chunks, each embedded on CPU. A reliable "it's really working" signal is your machine's CPU spiking — the n8n spinner alone can be misleading. Watch the insert count climb.*

> 🛑 ***Spinner stuck with CPU idle** for more than a minute or two? The n8n container has wedged (not your workflow). Fix and diagnosis in **Troubleshooting → symptom 3**.*

Verify the vectors landed (for this book, around 196):

```
curl -s http://localhost:6333/collections/documents | grep -o '"points_count":[0-9]*'
```

✅ **Part C done** when the run is green and `points_count` is greater than 0.

> 💡 ***Mental model:** the document is now "memorized" as vectors. In Part D we embed a question the same way, fetch the nearest chunks, and hand them to `llama3.2`.*

Part D — Chat with your document

You type a question → it's embedded → Qdrant returns the closest chunks → `llama3.2` answers grounded in them. This is a **separate workflow** from ingestion.

Node tree:

```
Chat Trigger -> Question and Answer Chain
                   |-- Model:     Ollama Chat Model (llama3.2)
                   |-- Retriever: Vector Store Retriever
                                     |-- Qdrant Vector Store (Retrieve mode,
collection 'documents')
                                                 |-- Embeddings Ollama (nomic-embed-text)
```

D1. New workflow + Chat Trigger — *do this in n8n*

1. **Create** → **Workflow**. Name it `Chat with docs`.
2. Add node → search **Chat Trigger** → under **Triggers**, add **On new Chat event**.

D2. Question and Answer Chain — *do this in n8n*

1. Click **+** after the Chat Trigger → search **Question and Answer Chain** → add it.
2. Leave the defaults — it exposes two connectors: **Model** and **Retriever**.

D3. Attach the chat model — *do this in n8n*

1. On the chain's **Model** connector → search **Ollama Chat Model** → add it.
2. **Credential:** your **Ollama account**.
3. **Model:** `llama3.2:latest`.

D4. Attach the retriever — *do this in n8n*

1. On the chain's **Retriever** connector → search **Vector Store Retriever** → add it.
2. It exposes a **Vector Store** connector → search **Qdrant Vector Store** → add it.
3. On that Qdrant node:
   - **Operation Mode:** `Retrieve Documents (As Vector Store for Chain/Tool)`.
   - **Credential: Qdrant account**.
   - **Qdrant Collection:** `documents`.

D5. Attach embeddings to the retriever's Qdrant node — *do this in n8n*

The query must be embedded with the **same** model used at ingestion.

1. On the Qdrant node's **Embeddings** connector → search **Embeddings Ollama** → add it.
2. **Credential: Ollama account**.
3. **Model:** `nomic-embed-text:latest`.

D6. Ask a question — *do this in n8n*

1. Click **Open chat** at the bottom of the canvas.
2. Ask something the book covers, e.g. *"What is the difference between bias and variance?"* or *"How should I set up my dev and test sets?"*
3. `llama3.2` answers using the retrieved chunks. The first answer is slower (the model loads into memory).

✅ **Part D done** when you get a grounded answer that reflects the document's content.

> 💡 ***The `*` is not an error.*** *The `*` on the **Model*** / **Retriever*** / **Vector Store*** connectors marks a required connection, not a problem. The Vector Store Retriever is a thin adapter and may not show its own green tick even when it ran. Judge success by the **Question and Answer Chain** node turning green and the answer reflecting your document.*

D7. Tune it: the "how much context?" experiment — *do this in n8n*

Don't just set the best value — **discover it.** This is the most important lesson in RAG.

First, run with the default. The **Vector Store Retriever** has a **Limit** of `4` (it returns the 4 closest chunks). Ask: *"What should I do to reduce avoidable bias in my learning algorithm?"* The book answers this across several short chapters — train a bigger model, train longer, add or modify input features, reduce regularization, and so on. With only 4 chunks, the small `llama3.2` model tends to give a **thin** answer, and because its default instructions tell it to say "I don't know" when unsure, it may even hedge with an *"I don't know … however …"* preamble.

Now raise the context. Open the **Vector Store Retriever** → set **Limit** to `8` → ask the same question again. With more of the relevant chapters in front of it, the answer fills out into a fuller list of techniques, and the hedging usually disappears.

The model, the document, and the question all stayed the same — the only change was going from 4 to 8 chunks of context.

> 🧪 *The exact wording changes every run (the model is non-deterministic) — watch the **pattern**: 4 chunks → thinner and more likely to hedge; 8 chunks → fuller and more confident.*

> 🎓 ***The lesson:*** *RAG has **two tunable halves** — retrieval (how much you fetch) and generation (the model). A weak answer isn't always a weak model. Always ask: "is it the search, or the model?" More levers — context length and the system prompt — are in **Tuning**.*

Part E — Log every Q&A to Postgres

Real RAG apps keep a record of what was asked and answered — for auditing, debugging, and building evaluation sets. We append each exchange to a Postgres table.

> 💡 *Our stack already runs Postgres (it's where n8n stores its own data). We add one small table to it — we don't install a database.*

E1. Create the log table (once)

Folder: `agentic-rag-n8n-cohort`. Apply the bundled SQL:

```
docker exec -i $(docker ps --format '{{.Names}}' | grep postgres) psql -U root -d n8n < sql/001_create_rag_chat_log.sql
```

This creates `rag_chat_log (id, question, answer, model, created_at)`.

E2. Add a Postgres credential in n8n — *do this in n8n*

1. http://localhost:5678/home/credentials → **Create credential** → search **Postgres**.
2. Fill in the container-internal values:

- **Host:** `postgres` ⚠️ n8n pre-fills `localhost` — change it to `postgres` (the container name).
- **Database:** `n8n` · **User:** `root` · **Password:** `password` · **Port:** `5432`
- **SSL:** `Disable`.

3. **Save.**

> ℹ️ *You may see **"No testing function found for this credential."** That's **not an error** — the Postgres credential simply has no built-in test. Save it anyway; the real test is E4.*

E3. Add the Postgres node to the chat workflow — *do this in n8n*

1. In **Chat with docs**, click **+** after the **Question and Answer Chain**.
2. Search **Postgres** → pick **Insert rows in a table**.
3. **Credential:** the one from E2. **Schema:** `public` · **Table:** `rag_chat_log` (`From list`).
4. **Mapping Column Mode:** `Map Each Column Manually`. n8n lists all 5 columns:
   - 🗑️ **Delete** `id` — Postgres auto-increments it; leaving `0` forces every row to id 0 and fails.
   - 🗑️ **Delete** `created_at` — Postgres fills it with `NOW()` automatically.
   - **question** → toggle to **Expression**, enter `{{ $('When chat message received').item.json.chatInput }}`
   - **answer** → toggle to **Expression**, enter `{{ $json.response }}`
   - **model** → leave as **Fixed**, type `llama3.2`

> ⚠️ *If a mapping shows empty, run the chat once, then click the field and pick the value from the **input panel** on the left — n8n inserts the correct expression for you.*

E4. Test it — *do this in n8n, then terminal*

1. Open chat → ask a question → wait for the answer.
2. Confirm the row landed:

```
docker exec $(docker ps --format '{{.Names}}' | grep postgres) psql -U root -d n8n -c "SELECT id, left(question,40), left(answer,50), created_at FROM rag_chat_log ORDER BY id DESC LIMIT 5;"
```

✅ **Part E done** when each chat question adds a row to `rag_chat_log`.

Part F — Your own web frontend via a webhook

So far you've chatted inside n8n. Now we expose the RAG pipeline as a **webhook** — a URL n8n listens on — so any app can use it, and point a small React page at it.

> 💡 *A webhook is just an HTTP endpoint. Your page POSTs a question to it; n8n runs the same RAG chain and POSTs the answer back. This is how real apps wrap an AI backend.*

Node tree (a **third** workflow):

```
Webhook (POST) -> Question and Answer Chain -> Respond to Webhook
                    |-- Ollama Chat Model (llama3.2)
                    |-- Vector Store Retriever -> Qdrant -> Embeddings Ollama
```

F1. Build the webhook workflow — *do this in n8n*

1. **Create** → **Workflow**, name it `RAG webhook`.
2. Add a **Webhook** node:
   - **HTTP Method:** `POST`
   - **Path:** `rag-chat` (the URL becomes `…/webhook/rag-chat`)
   - **Respond:** `Using 'Respond to Webhook' Node`
   - **Options → Allowed Origins (CORS):** `*` ⚠️ **required** — without it the browser blocks the cross-origin call from the page.
3. Add a **Question and Answer Chain** after the Webhook:
   - **Source for Prompt (User Message):** change to `Define below`.
   - **Prompt (User Message)** field (set to **Expression**): `{{ $json.body.question }}`
   - Attach the **same** sub-nodes as Part D: **Ollama Chat Model** (`llama3.2`) on Model, and **Vector Store Retriever → Qdrant (Retrieve) → Embeddings Ollama** on Retriever.
4. Add a **Respond to Webhook** node after the chain:
   - **Respond With:** `JSON`
   - **Response Body** (toggle to **Expression**): `{ "answer": {{ JSON.stringify($json.response) }} }`

F2. Publish the workflow — do this FIRST — *do this in n8n*

Your page calls the webhook over HTTP, so the workflow must be **live**:

1. Click **Save**, then toggle the workflow **Active** (top-right). That is what "publishing" means.
2. Open the **Webhook** node — it now shows a **Production URL**: `http://localhost:5678/webhook/rag-chat`. Copy it.

> ⚠️ *Skip this and the page shows **"Failed to fetch" / 404** — an inactive workflow has no live production URL.*

F3. Run the page

The page runs on your **laptop** (Node/Vite), not in a container. Start its dev server:

Folder: `agentic-rag-n8n-cohort/frontend`.

```
cd frontend
```

```
npm install
```

```
npm run dev
```

It serves the page at `http://localhost:5173`.

> 💡 ***No Node?*** *If `npm` is missing or `node -v` is below 18, install Node first — see **Troubleshooting → symptom 5**.*

F4. Test it

1. The shipped page already points `WEBHOOK_URL` at the Production URL from F2, so usually no change is needed. (To change it, edit `frontend/src/App.jsx`.)
2. Open http://localhost:5173, type a question → **Ask** → the answer appears and is added to the chat history below.

> ✅ ***The order that makes it work:*** *(1) workflow* ***Active*** *(F2) → (2)* `npm run dev` *→ (3) ask. If you hit "Failed to fetch", re-check that the workflow is* ***Active*** *and* ***CORS =*** `*` *on the Webhook node.*

✅ **Part F done** when the page shows an answer pulled from your document and keeps a chat history.

> 💡 ***Build it yourself (optional exercise).*** *The shipped* `frontend/src/App.jsx` *is a reference answer. To build your own, prompt your AI IDE with: the stack (React + Vite, plain JS), the contract (POST* `{ "question": "..." }` *→* `{ "answer": "..." }`*), the UI (input + Ask button on top, chat history below), and the environment (page at :5173, webhook at :5678, CORS may need enabling). The request/response contract is the part most people forget — and it's what makes the page connect.*

The payoff — what you just built

You now have a complete, fully-local RAG application: it ingests real documents, answers questions grounded in them, logs every exchange, and serves answers to your own web page — all on your machine, no cloud.

The single most important thing you learned is the **4 → 8 context experiment** in Part D: the same model gave a thinner answer with 4 retrieved chunks and a fuller one with 8. Nothing about the model changed — only *how much of your document it was shown*. **RAG has two tunable halves — retrieval and generation.** When an answer disappoints, ask first: *is it the search, or the model?*

Where to go next

- **Ingest your own corpus.** Drop your own `.txt`, `.pdf`, `.csv`, `.xlsx`, or `.html` into `shared/corpus/` and re-run ingestion. (Word `.docx`? Save as PDF first.) Re-ingesting a fresh document? Clear the collection first (delete + recreate it, B4) so old vectors don't linger.
- **Improve weak answers.** Raise the retriever Limit, the model's context length, and replace the default system prompt — the full walkthrough is in **Tuning**.
- **Try a bigger model.** Pull `mistral` and switch the Ollama Chat Model; compare its confidence to `llama3.2`.
- **Build an eval set.** Query `rag_chat_log` to review past answers, flag bad ones, and turn them into test questions — the start of measuring, not guessing, whether changes help.

> 🆘 ***Something not working?*** *Every common problem — file-access errors, the wedged-container spinner, a missing model, "vite: command not found", webhook "Failed to fetch", weak answers — is covered with an exact fix in* ***Troubleshooting & Tuning****.*